QALight

Frontend-розробка

Заняття 23. TypeScript

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



TypeScript – вступ та початок роботи.
Налаштування середовища розробки TypeScript.
Компіляція TypeScript в JavaScript. Файл конфігурації TypeScript.
Встановлення Node.js та npm. Встановлення TypeScript Compiler (TSC).

TypeScript

Налаштування середовища розробки TypeScript. Компіляція TypeScript в JavaScript. Файл конфігурації TypeScript

TypeScript – вступ та початок роботи

Теорія: TypeScript

TypeScript - це мова програмування, яка являє собою надмножину JavaScript. Вона додає **статичну типізацію** і деякі додаткові функції, які допомагають розробникам створювати більш надійні та масштабовані додатки. TypeScript зберігає сумісність зі стандартами JavaScript, що робить її легко інтегрованою в наявні проєкти.

- TypeScript додає до JavaScript систему типів, яка допомагає писати код, що читається легше, менше схильний до помилок, і краще організований.
- TypeScript використовується в багатьох популярних проєктах. Наприклад, Angular, React Native, Babel, Jest, і багато інших. Це свідчить про те, що TypeScript – це мова, яка затребувана в реальному світі.
- TypeScript може допомогти вам краще зрозуміти JavaScript. Система типів TypeScript змушує вас чітко описувати, що робить ваш код.
- TypeScript має велику та активну спільноту. Це означає, що ви можете знайти багато інформації про TypeScript в Інтернеті.
- TypeScript – це мова, яка постійно розвивається.
- TypeScript – це мова, яка може допомогти вам у вашій кар'єрі. Багато компаній шукають розробників, які володіють TypeScript. Вивчення TypeScript може дати вам конкурентну перевагу на ринку праці.

Різниця між TypeScript та JavaScript

Теорія: TypeScript

TypeScript - це надмножина JavaScript, що означає, що ви можете використовувати весь ваш наявний JavaScript-код без змін. Він додає статичну типізацію, яка допомагає виявляти помилки на етапі компіляції та створювати надійніші додатки. Однак, для роботи з TypeScript потрібен деякий час для вивчення і звикання до його особливостей.

JavaScript, з іншого боку, є базовою мовою програмування для веб-розробки. Вона має динамічну типізацію, що означає, що типи змінних визначаються автоматично під час виконання. JavaScript надає великий вибір інструментів і бібліотек, а також дає змогу швидко прототипувати та розробляти додатки.

Незалежно від вибору, обидві мови мають широку підтримку інструментів розробки та активні спільноти розробників. Це забезпечує доступ до навчальних матеріалів, документації та допомоги за потреби.

Навіщо потрібен TypeScript?

Теорія: TypeScript

Великі проєкти та командна розробка:

У команді розробників працює кілька людей над різними модулями веб-додатка. Завдяки статичній типізації TypeScript, кожен розробник може чітко розуміти очікувану структуру даних і типи аргументів функцій, що спрощує спільну роботу і запобігає можливим помилкам інтеграції.

Ваш веб-додаток був написаний на JavaScript і має безліч функцій і компонентів. Ви вирішуєте перейти на TypeScript і починаєте додавати типи до наявних функцій і змінних. Це допомагає виявити потенційні помилки в коді та спростити його підтримку, оскільки типи надають чітке розуміння очікуваної структури даних.

Інтеграція з фреймворками та бібліотеками: Ви розробляєте веб-додаток на Angular використовуєте TypeScript. Завдяки типізованим оголошенням для Angular, ви отримуєте підказки типів під час розроблення компонентів, що допомагає уникнути помилок і забезпечує надійнішу взаємодію між компонентами.

Покращення продуктивності та надійності: У вашому додатку є складна логіка, що включає безліч функцій і класів. Використовуючи TypeScript, ви можете визначити типи вхідних і вихідних даних для кожної функції та методу, що допомагає запобігти помилкам під час використання неправильних аргументів або невірних типів даних.

Налаштування середовища розробки

Практика: TypeScript

Налаштування середовища розробки для TypeScript складається з кількох простих кроків:

- встановлення Node.js
- ініціалізації проєкту
- встановлення TypeScript
- налаштування компілятора

Кросплатформне середовище виконання JS Node.js

Теорія: TypeScript

https://nodejs.org/uk - це кросплатформне середовище виконання JavaScript з відкритим кодом, яке дозволяє запускати код JavaScript поза межами вебпереглядача (на комп'ютері чи сервері).

Детальну інформацію та документацію можна знайти на офіційному сайті Node.js.

Головні особливості

- Двигун V8: Побудований на рушійі JavaScript V8 від Google, який перетворює код JavaScript безпосередньо на швидкий машинний код.
- Асинхронність та неблокуюче введення-виведення: Працює за принципом обробки подій, що дозволяє ефективно виконувати безліч операцій одночасно без зависань.
- Менеджер пакетів npm: Має величезну бібліотеку готових модулів та інструментів.]

Для чого використовується

- Створення швидких вебсерверів та бекенду (серверної частини) застосунків.
- Розробка чатів у реальному часі, API та стрімінгових сервісів.
- Написання скриптів та утиліт для командного рядка (CLI).

TypeScript

Встановлення Node.js та npm.

Встановлення Node.js та npm

Практика: TypeScript

TypeScript працює поверх платформи Node.js. Перевірте, чи встановлені вони у вас, виконавши команди в терміналі:

```
node -v
npm -v
```

Якщо команди не знайдено, завантажте та встановіть Node.js (рекомендовану версію LTS) з офіційного сайту nodejs.org.

- Завантажте та встановіть офіційну версію Node.js з її сайту (разом із нею встановиться пакетний менеджер npm).
- Відкрийте термінал (командний рядок) і перевірте встановлення командами:

```
node -v
npm -v
```

Встановлення Node.js та npm (Linux)(

Практика: TypeScript

Завантажує й установлює nvm:
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash

Замість перезапуску оболонки можна виконати:
\. "$HOME/.nvm/nvm.sh"

Завантажує й установлює Node.js:
nvm install 24

Перевіряє версію Node.js:
node -v # Повинно вивести «v24.21.0».

Перевіряє версію npm:
npm -v # Повинно вивести «11.19.0».
nvm — це кросплатформний менеджер версій Node.js. Якщо виявите проблеми, відвідайте вебсайт [nvm](https://github.com/nvm-sh/nvm)

Встановлення Node.js та npm (Windows)

Практика: TypeScript

Завантажує й установлює Chocolatey:
powershell -c "irm https://community.chocolatey.org/install.ps1|iex"

Завантажує й установлює Node.js:
choco install nodejs --version="24.21.0"

Перевіряє версію Node.js:
node -v # Повинно вивести «v24.21.0».

Перевіряє версію npm:
npm -v # Повинно вивести «11.19.0».

Chocolatey — це менеджер пакетів для Windows. Якщо виявите проблеми, відвідайте вебсайт Chocolatey (https://chocolatey.org/)

Встановлення Node.js та npm (macOs)

Практика: TypeScript

Завантажує й установлює nvm:
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash

Замість перезапуску оболонки можна виконати:
\. "$HOME/.nvm/nvm.sh"

Завантажує й установлює Node.js:
nvm install 24

Перевіряє версію Node.js:
node -v # Повинно вивести «v24.21.0».

Перевіряє версію npm:
npm -v # Повинно вивести «11.19.0».
nvm — це кросплатформний менеджер версій Node.js. Якщо виявите проблеми, відвідайте вебсайт [nvm](https://github.com/nvm-sh/nvm)

TypeScript

Встановлення TypeScript Compiler (TSC)

Компілятор TypeScript

Практика: TypeScript

Компілятор TypeScript - це інструмент, який перетворює код TypeScript на звичайний JavaScript. Він виконує перевірку типів, а потім генерує еквівалентний JavaScript-код, який можна запустити в будь-якому середовищі виконання, сумісному з JavaScript.

Компілятор TypeScript також дає змогу налаштувати різні параметри компіляції, як-от цільову версію JavaScript, рівень суворої перевірки типів та інші опції. Це дає змогу розробникам налаштовувати компіляцію відповідно до вимог та особливостей свого проєкту.

Загалом, компілятор є важливим компонентом у процесі розробки на TypeScript. Він забезпечує перевірку типів, перетворення коду в JavaScript і налаштування параметрів компіляції, що робить TypeScript потужним інструментом для створення надійних і масштабованих додатків.

Встановлення TypeScript Compiler (TSC)

Практика: TypeScript

Встановіть TypeScript як залежність для розробки (**devDependencies**):
npm install typescript --save-dev

Також рекомендується встановити пакети типів для Node.js, щоб TypeScript розумів вбудовані модулі (наприклад, fs, path):
npm install @types/node --save-dev

Глобальне встановлення: Встановіть компілятор tsc за допомогою термінала:
npm install -g typescript

Перевірка версії:
tsc -v

Компілятор встановлено в каталозі **node_modules** і може працювати за допомогою: **npx tsc**.
npx tsc

Створення та ініціалізація проєкту

Практика: TypeScript

Створіть нову папку для вашого проєкту, перейдіть у неї та створіть файл конфігурації **package.json**:

```
mkdir ts-base
cd ts-base
npm init -y
```

```
{
 "name": "ts-base",
 "version": "1.0.0",
 "description": "",
 "main": "index.js",
 "scripts": {
   "test": "echo \"Error: no test specified\" && exit 1"
 },
 "keywords": [],
 "author": "",
 "license": "ISC",
 "type": "commonjs",
 "dependencies": {
   "typescript": "^7.0.2"
 }
}
```

Створення файлу конфігурації tsconfig.json

Файл tsconfig.json визначає правила компіляції коду.
Створіть файл конфігурації tsconfig.json за допомогою команди:

npx tsc --init

Цей файл керує параметрами компіляції та структурою вашого проєкту.

Created a new tsconfig.json

TS

You can learn more at https://aka.ms/tsconfig

Файл конфігурації TypeScript

Теорія: TypeScript

Відкрийте створений файл tsconfig.json та розкоментуйте або змініть такі ключові параметри для зручної структури проєкту:

```
{
 // Visit https://aka.ms/tsconfig to read more about this file
 "compilerOptions": {
   // File Layout
   // "rootDir": "./src",
   // "outDir": "./dist",

   // Environment Settings
   // See also https://aka.ms/tsconfig/module
   "module": "nodenext",
   "target": "esnext",
   "types": [],
   // For nodejs:
   // "lib": ["esnext"],
   // "types": ["node"],
   // and npm install -D @types/node
```

Файл конфігурації TypeScript

Теорія: TypeScript

```
// Other Outputs
"sourceMap": true,
"declaration": true,
"declarationMap": true,

// Stricter Typechecking Options
"noUncheckedIndexedAccess": true,
"exactOptionalPropertyTypes": true,
```

Файл конфігурації TypeScript

Теорія: TypeScript

```
    // Style Options
    // "noImplicitReturns": true,
    // "noImplicitOverride": true,
    // "noUnusedLocals": true,
    // "noUnusedParameters": true,
    // "noFallthroughCasesInSwitch": true,
    // "noPropertyAccessFromIndexSignature": true,

    // Recommended Options
    "strict": true,
    "jsx": "react-jsx",
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "noUncheckedSideEffectImports": true,
    "moduleDetection": "force",
    "skipLibCheck": true,
  }}
```

Налаштування компілятора

Практика: TypeScript

Компілятор можна налаштувати за допомогою файлу tsconfig.json.

За замовчуванням компілятор TypeScript друкуватиме довідкове повідомлення під час запуску в порожньому проєкті.

Приклад того, що ви можете додати до файлу tsconfig.json:

```
"compilerOptions": {
  "target": "esnext",                    // Версія JavaScript, в яку компілюється код
  "module": "NodeNext",                  // Сучасна система модулів
```

Параметр target

Практика: TypeScript

Сучасні браузери підтримують усі можливості ES6, тому ES6 є вдалим вибором. Ви можете встановити нижчу цільову версію (**target**), якщо код розгортається в застарілих середовищах, або вищу, якщо гарантовано використовуватимете його в новіших середовищах.
Налаштування `target` визначає, які можливості JavaScript будуть перетворені на старіші аналоги (downleveling), а які залишаться без змін.
Наприклад, стрілкова функція `() => this` буде перетворена на еквівалентний вираз функції, якщо для `target` встановлено значення ES5 або нижче.
Зміна `target` також впливає на значення параметра **`lib`** за замовчуванням. Ви можете комбінувати налаштування `target` і `lib` на власний розсуд, хоча для зручності часто достатньо налаштувати лише `target`. Для платформ розробки, як-от Node, існують стандартні значення `target`, що залежать від типу платформи та її версії.
Спеціальне значення **`ESNext` відповідає найновішій версії**, яку підтримує версія TypeScript. Цей параметр слід використовувати з обережністю, оскільки його значення може відрізнятися в різних версіях TypeScript, що робить процес оновлення менш передбачуваним.
Починаючи з TypeScript 6.0, значенням за замовчуванням є `es2025`, яке оновлюється відповідно до розвитку стандарту ECMAScript.

Параметр module

Практика: TypeScript

Параметр `module` визначає систему модулів для програми. Найімовірніше, для сучасних проєктів на Node.js вам знадобиться значення `nodenext`, а для коду, що підлягатиме збиранню в бандл (bundling), - `preserve` або `esnext`.

Зміна параметра `module` впливає на `moduleResolution` .

```
"compilerOptions": {
  "target": "esnext",                    // Версія JavaScript, в яку компілюється код
  "module": "NodeNext",                  // Сучасна система модулів
```

Параметр moduleDetection

Практика: TypeScript

параметр moduleDetection визначає, як TypeScript розпізнає, чи є файл скриптом або модулем.

Доступні три варіанти:

- `"auto"` (за замовчуванням) - TypeScript не лише шукає інструкції `import` та `export`, а й перевіряє, чи встановлено поле `"type"` у файлі `package.json` як `"module"` (за умови використання `module`: `nodenext` або `node16`), а також перевіряє, чи є поточний файл JSX-файлом (за умови використання `jsx`: `react-jsx`).
- `"legacy"` - Поведінка, ідентична версії 4.6 та попереднім: для визначення того, чи є файл модулем, використовуються інструкції `import` та `export`.
- `"force"` - Забезпечує обробку кожного файлу (окрім файлів оголошень) як модуля.

Структура файлів

Практика: TypeScript

```
// Visit https://aka.ms/tsconfig to read more about this file
"compilerOptions": {
  // File Layout
  "rootDir": "./src",
  "outDir": "./dist",
```

За замовчуванням: **rootDir** - каталог, що містить файл tsconfig.json.

Визначене значення для `rootDir` - це найдовший спільний шлях усіх вхідних файлів (окрім файлів оголошень); у цьому випадку це `src/`.

Каталог outDir

Практика: TypeScript

```
// Visit https://aka.ms/tsconfig to read more about this file
 "compilerOptions": {
   // File Layout
   "rootDir": "./src",
   "outDir": "./dist",
```

Якщо вказано outDir, файли .js (а також .d.ts, .js.map тощо) будуть записані в цю директорію.

Структура директорій вихідних файлів зберігається;

Створення структури папок

Практика: TypeScript

Створіть папку **src** та файл **index.ts** всередині неї:

mkdir src
touch src/index.ts

Додайте простий код у **src/index.ts:**

```
const message: string = "Середовище TypeScript успішно налаштовано!";
console.log(message);
```

Компіляція та запуск коду

Практика: TypeScript

Щоб скомпілювати TypeScript у JavaScript, виконайте команду:

npx tsc

У проєкті з'явиться папка dist з файлом index.js. Його можна запустити за допомогою Node.js:

node dist/index.js

Інструменти для автоматизації

Практика: TypeScript

Для автоматизації розробки в TypeScript найчастіше налаштовують **автоматичний перезапуск коду при зміні файлів** (Hot Reload) та зручні **скрипти автоматизації у** `package.json`.
Автоматична перекомпіляція:

Найкращим вибором для автоматизації є зв'язка `tsx` (сучасний і дуже швидкий запуск) + `tsx --watch` або використання інструменту `nodemon`.

Щоб компілятор стежив за змінами у файлах і автоматично оновлював JS, запустіть команду в режимі "**watch**":

npx tsc -w

Скрипти автоматизації у файлі package.json

Практика: TypeScript

Додайте скрипти автоматизації у файл `package.json`:
Відкрийте `package.json` та замініть секцію `"scripts"` на таку:

```
"scripts": {
  "dev": "tsx watch src/index.ts",
  "build": "tsc",
  "start": "node dist/index.js"
}
```

Як це працює:

- `npm run dev` - запускає проєкт у режимі розробки. При будь-якій зміні коду в папці `src` проєкт автоматично перезапуститься в терміналі за мілісекунди.
- `npm run build` - компілює фінальну (production) версію коду в папку `dist`.
- `npm run start` - запускає вже скомпільований JavaScript (використовується на сервері).

Скрипт автоматизації за допомогою nodemon

Практика: TypeScript

Якщо ви віддаєте перевагу перевіреному часом nodemon (наприклад, для складних Node.js/Express серверів), його можна поєднати з компілятором tsc.

1. Встановіть nodemon:

```
npm install nodemon --save-dev
```

2. Створіть файл конфігурації nodemon.json у корені проєкту:

```
{
  "watch": ["src"],
  "ext": ".ts,.js",
  "ignore": [],
  "exec": "npx tsc && node dist/index.js"
}
```

3. Додайте скрипт у package.json:

```
"scripts": {
  "dev": "nodemon"
}
```

Тепер команда npm run dev буде автоматично перекомпілювати код та перезапускати сервер при кожному збереженні файлу.

Автоматичний лінтинг (ESLint)

Практика: TypeScript

Для автоматичного лінтингу TypeScript-коду найкраще використовувати **ESLint**.

1. Встановіть ESLint:
Якщо ви ще не встановили пакет, зробіть це через термінал:
npm install eslint --save-dev

2. Розташування файлу:
Файл обов'язково має лежати в корені проєкту (там же, де і package.json).

3. Експорт масиву:
Формат (eslint.config.js) завжди експортує масив об'єктів конфігурації замість старих об'єктів module.exports.

Файл eslint.config.js

Практика: TypeScript

```
import js from "@eslint/js";

export default [
 js.configs.recommended,
 {
   files: ["**/*.js"],
   languageOptions: {
     ecmaVersion: "latest",
     sourceType: "module",
     globals: {
       // Додайте глобальні змінні за потреби (наприклад, для браузера чи node)
       window: "readonly",
       document: "readonly",
       process: "readonly",
     },
   },
```

Файл eslint.config.js

Практика: TypeScript

```
    rules: {
      "no-unused-vars": "warn",
      "no-console": "off",
    },
  },
  {
    // Глобальне ігнорування папок
    ignores: ["node_modules/", "dist/", "build/"],
  }
];
```

Налаштування автоматичного форматування коду

Практика: TypeScript

Щоб код автоматично вирівнювався та відповідав єдиному стилю, варто додати Prettier.
1. Встановіть Prettier:
Якщо ви ще не встановили сам Prettier, додайте і його: npm install --save-dev prettier
npm install --save-dev eslint-config-prettier

2. Додайте скрипт форматування у package.json:

```
"scripts": {
  "dev": "tsx watch src/index.ts",
  "build": "tsc",
  "start": "node dist/index.js",
  "format": "prettier --write \"src/**/*.ts\""
}
```

Тепер команда npm run format автоматично виправить усі відступи та крапки з комою у ваших файлах.

Пакет typescript-eslint

Практика: TypeScript

Для підтримки TypeScript у новому форматі ESLint (Flat Config) використовується офіційний пакет typescript-eslint.

Виконайте команду у терміналі, щоб встановити плагін та парсер для TypeScript:

npm install --save-dev typescript-eslint @eslint/js eslint

Офіційний стандартний спосіб створювати конфігурації у новому Flat Config форматі - це використовувати вбудовану функцію defineConfig безпосередньо з пакету eslint/config.

Файл конфігурації

Практика: TypeScript

Оновіть ваш конфігураційний файл. Конфігурація Prettier обов'язково має йти останньою у списку, щоб перевизначити попередні правила:

```
import js from "@eslint/js";
import tseslint from "typescript-eslint";
import eslintConfigPrettier from "eslint-config-prettier";
import { defineConfig } from "eslint/config"; // Імпортуємо новий стандартний хелпер

export default defineConfig([

 // 1. Базові рекомендовані правила JavaScript та TypeScript
 js.configs.recommended,
 ...tseslint.configs.recommended,
```

Файл конфігурації

Практика: TypeScript

```
  // 2. Ваші кастомні налаштування правил для TS
{
  files: ["**/*.ts", "**/*.tsx"],
  languageOptions: {
    parser: tseslint.parser,
    parserOptions: {
      ecmaVersion: "latest",
      sourceType: "module",
    },
  },
  rules: {
    "@typescript-eslint/no-unused-vars": "warn",
    "@typescript-eslint/no-explicit-any": "warn",
    "no-console": "off",
  },
},
```

Файл конфігурації

Практика: TypeScript

```
  // 3. Вимкнення конфліктних правил Prettier (має бути в самому кінці)
 eslintConfigPrettier,
  // 4. Глобальне ігнорування папок
 {
   ignores: ["node_modules/", "dist/", "build/", "eslint.config.js"],
 }
]);
```

Тестування коду на TypeScript

Практика: TypeScript

Для тестування коду на TypeScript найкращим вибором є **Vitest**. Він створений на базі надшвидкого збирача Vite, працює в рази швидше за застарілий Jest і підтримує TypeScript "із коробки" без складних додаткових конфігурацій.

1. Встановлення пакетів

Встановіть **Vitest** у якості залежності розробки:

```
npm install vitest --save-dev
```

Тестування коду на TypeScript

Практика: TypeScript

2. Додавання скриптів у package.json
Додайте команди для запуску тестів у секцію "scripts" вашого файлу **package.json**:

```
"scripts": {
  "dev": "tsx watch src/index.ts",
  "build": "tsc",
  "start": "node dist/index.js",
  "format": "prettier --write \"src/**/*.ts\"",
  "lint": "eslint .",
  "lint:fix": "eslint . --fix",
  "test": "vitest",
  "test:run": "vitest run"
}
```

- npm run test - запускає тести у **режимі watch**. Vitest буде стежити за змінами у коді та тестах і миттєво перезапускати лише потрібні перевірки.
- npm run test:run - запускає тести **один раз** (корисно для перевірки перед деплоєм або в CI/CD).

]

Створення тесту

Практика: TypeScript

3. Створення тесту

Крок 3.1. Створіть файл з логікою src/math.ts:

```
export function add(a: number, b: number): number {
  return a + b;
}
```

Крок 3.2. Створіть файл тесту src/math.test.ts:

Vitest автоматично шукає файли з розширенням .test.ts або .spec.ts.

```
import { describe, it, expect } from 'vitest';
import { add } from './math';

describe('Функція add', () => {
  it('має правильно додавати два додатні числа', () => {
    expect(add(2, 3)).toBe(5);
  });
  it('має правильно працювати з від’ємними числами', () => {
    expect(add(-1, 1)).toBe(0);
  });
});
```

Запуск тестів

Практика: TypeScript

Виконайте команду в терміналі:

`npm run test`

Ви побачите гарний та швидкий звіт про успішне проходження тестів прямо в консолі.

Налаштування звітів про покриття коду (**Code Coverage**)
Якщо ви хочете бачити, який відсоток вашого коду покритий тестами, Vitest має вбудовану підтримку покриття через інструмент `v8`.

1. Встановіть інструмент покриття:

`npm install @vitest/coverage-v8 --save-dev`

1. Додайте скрипт у `package.json`:

`"test:coverage": "vitest run --coverage"`

Тепер запуск `npm run test:coverage` покаже детальну таблицю у терміналі та створить папку `coverage` з інтерактивним HTML-звітом.

ВПРАВИ

Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Typescriptlang: https://www.typescriptlang.org/

Tsconfig: https://www.typescriptlang.org/tsconfig/

Microsoft for JavaScript developers: https://developer.microsoft.com/en-us/javascript/

JavaScript Guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Nodejs:](https://nodejs.org/uk)

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua